01 Implementation intent and scope

Andalusia Academy
Phase 1 Implementation Plan

Technical architecture and delivery specification

Version 1.0 | 10 September 2026 | Proposed implementation baseline

I will build Phase 1 as a modular monolith using ASP.NET Core, EF Core code-first migrations, Microsoft SQL Server, and React with TypeScript. My objective is to deliver the full visitor-to-learner journey with clear module boundaries, reliable transactions, enforceable authorization, and measurable performance.

This plan defines how I intend to implement the platform. It covers architecture, logical ERDs, entity responsibilities, business invariants, frontend behavior, migration strategy, operations, testing, and delivery gates. Performance figures are validation targets; they are not benchmark results or delivery commitments.

Launch boundary

The baseline is the Andalusia Academy Phase 1 MVB scope, version 1.0 dated 19 August 2026. Phase 1 includes public discovery, registration, applications, payment, enrollment, basic learning delivery, learner and instructor dashboards, management operations, content editing, and notifications. [1]

- Public content: homepage, About Us, course and program catalogs, offline course details, career paths, program comparison, corporate training information, instructor profiles, partners, accreditation, FAQ, testimonials, contact and support.
- Learning: course access, schedules and locations, private resources, attendance, assignments, and basic progress.
- Operations: learner and instructor management, KPIs, reports, permissions, instructor compensation, payouts, recent activity, and editable content.

Deferred scope

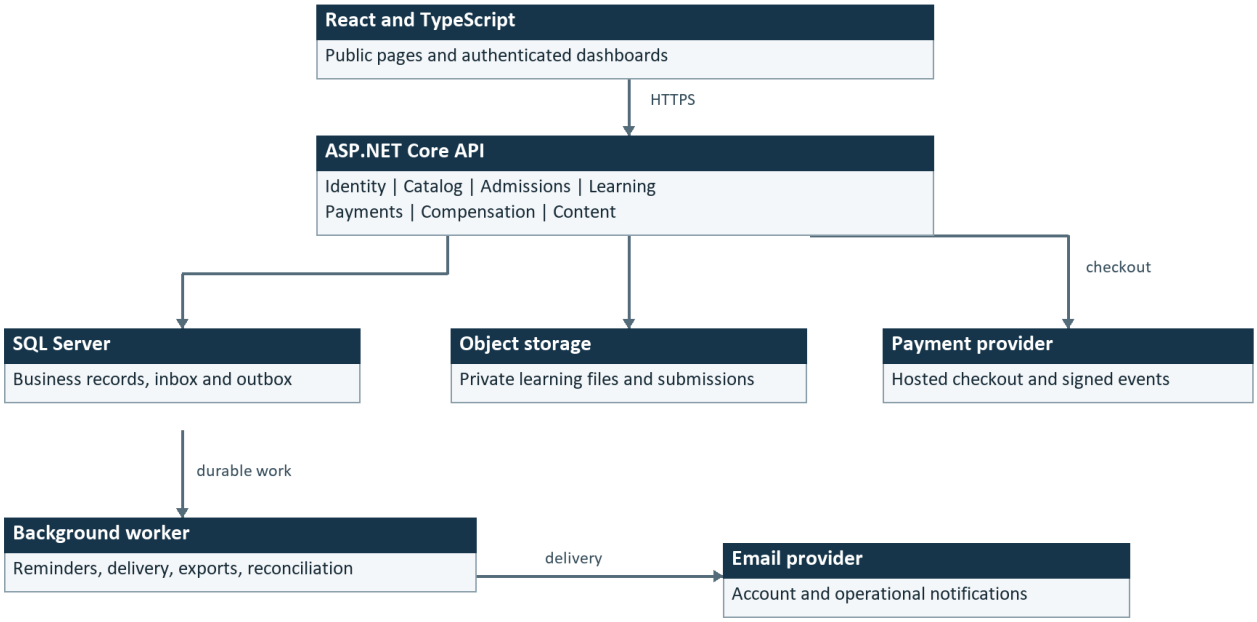
I will keep direct messaging, calendar functionality beyond required schedule views, quizzes, assessments, question banks, advanced grading, certificates, AI features, gamification, advanced analytics, alumni services, and advanced automation outside Phase 1. Corporate account administration and bulk enrollment belong to a later phase; the public corporate training page remains in scope. [1]

First delivery target

My first end-to-end slice is: publish a course offering, submit an application, complete a verified payment, create enrollment, and access a private learning resource. This validates the highest-risk dependencies before expanding the dashboards.

02 System architecture

I will deploy one API application with explicit business modules, one SQL Server database, and a separately runnable background worker. The API and worker can scale independently. I will retain transactional consistency inside the monolith and introduce additional infrastructure only when measurements justify it. [2]



Payment webhooks return to a dedicated API endpoint. The API verifies and persists them before acknowledging receipt. Arrows in this overview show primary dependencies rather than every request and response.

Component	Baseline
Runtime	.NET 10 LTS, ASP.NET Core 10, EF Core 10; pin compatible current patches. [3]
Frontend	React and TypeScript; prerender public pages and client-render authenticated workflows. [4]
Database	SQL Server; schemas by module and one application migration stream.
Integrations	Provider adapters for payment, email, storage, and durable background work.

03 Module boundaries and frontend design

Module	Owned responsibilities
Identity	Accounts, confirmation, password reset, profile, roles, permissions, consent.
Catalog and content	Courses, programs, career paths, categories, publication and public content.
Admissions	Applications, admission decisions, offering capacity, reservations, enrollment.
Learning	Course participation, sessions, instructor assignments, resources, submissions, attendance and progress.
Payments	Orders, payment attempts, verified events, reconciliation and approved refund workflow.
Compensation	Agreements, earning entries, approval and payout allocations.
Communications and operations	Announcements, notification deliveries, outbox processing, audit trail and reports.

Internal structure

I will separate domain rules, application use cases, infrastructure adapters, and HTTP endpoints. Application features will be organized around use cases such as Apply, ConfirmPayment, EnrollLearner, RecordAttendance, and ApprovePayout. I will avoid a large generic service layer and a generic repository that merely duplicates DbContext behavior.

One application DbContext will use entity configurations grouped by module. Each module owns its writes. Cross-module operations are explicit application workflows with short transactions. External integrations sit behind narrow interfaces. I will use domain events where useful and an outbox for durable external side effects; event sourcing is not required.

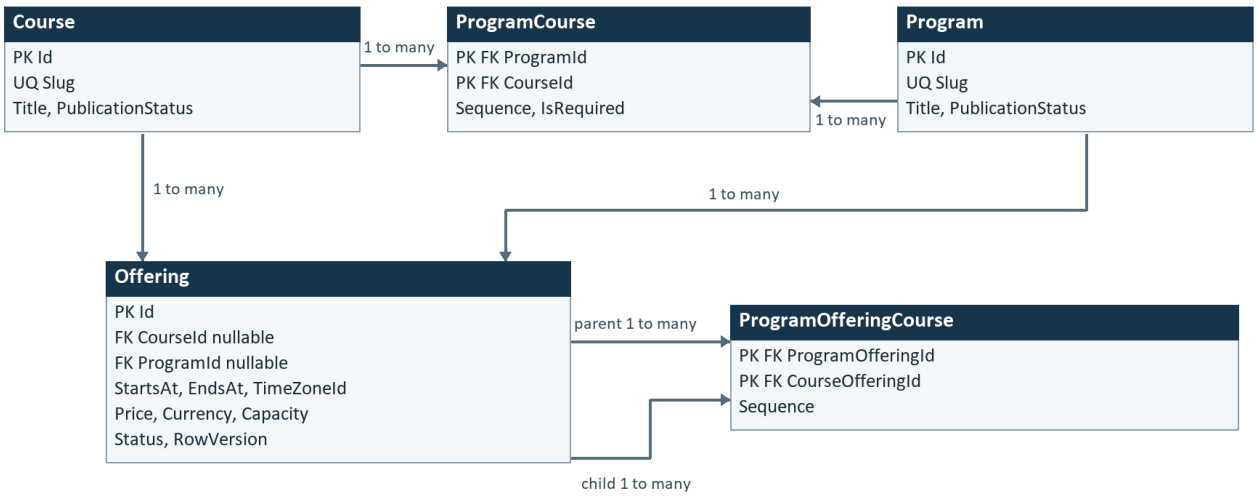
React application behavior

I will use feature-based folders, typed API contracts, shared accessible components, and a query cache for server state. Form state and UI state stay local unless they are genuinely shared. API DTOs are separate from EF entities. Public pages use prerendering and publish-triggered refresh; learner-specific responses must never enter a shared public cache.

- Routes cover public discovery, authentication, learner dashboard, instructor workspace, manager/admin operations, and content editing.
- Lists support filtering, stable sorting and server pagination. Forms expose validation, loading, failure, retry and empty states. Payment return pages display verification status from the API.
- Role-aware navigation improves usability; the API enforces permissions independently. Dates display in the intended timezone. Preferred language is stored; bilingual content and RTL remain a launch decision.
- Same-origin hosting routes API calls under a common origin. Cookie authentication uses Secure and HttpOnly attributes and CSRF protection for state-changing requests.

04 Catalog and offering ERD

A Course is a reusable catalog definition. An Offering is a scheduled intake with its own price, capacity and status. A Session is an actual meeting. A Program is a reusable curriculum; its offering binds that curriculum to actual course offerings.



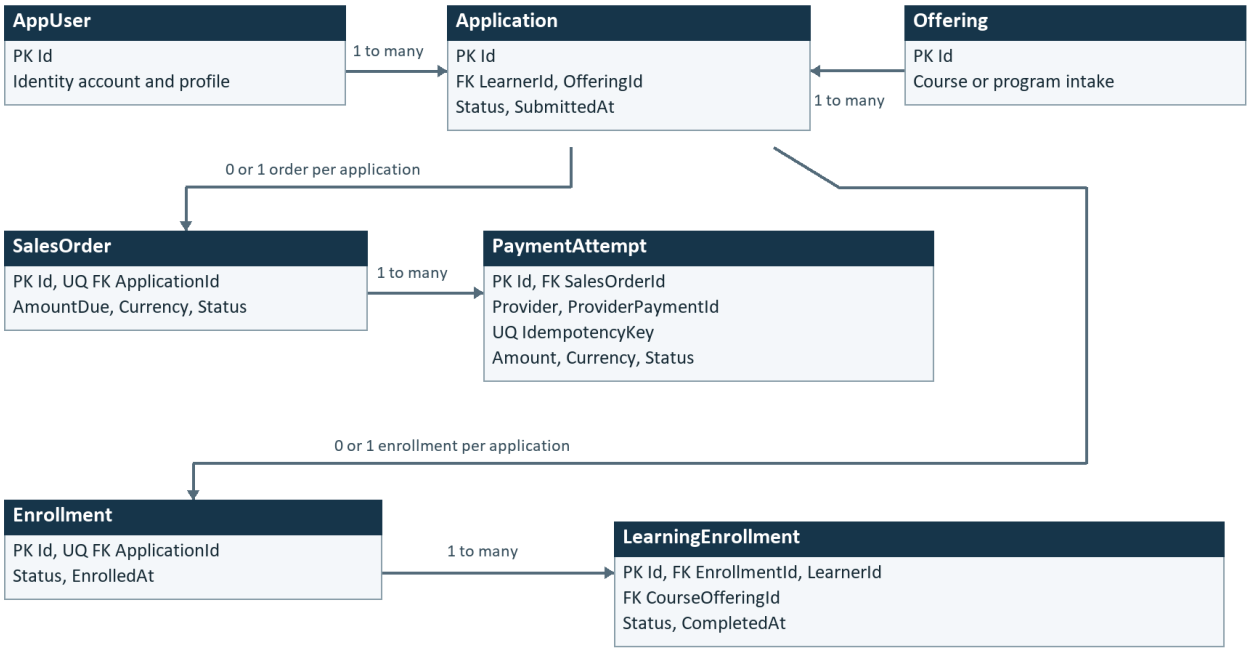
Notation: PK primary key; FK foreign key; UQ unique. Relationship labels state the cardinality.

Model invariants

- Offering.CourseId and Offering.ProgramId are mutually exclusive: exactly one is non-null. Both are real foreign keys; a SQL check constraint enforces the exclusive choice.
- ProgramOfferingCourse requires a program offering as parent and a course offering as child. Application validation verifies types and curriculum membership.
- Published course descriptions can evolve, but the included offerings and commercial terms of an enrolled intake must not change silently. I will freeze intake composition when enrollment opens or use an explicit transfer/change workflow.
- Course and program slugs are unique within their route namespaces. Categories and career paths use explicit join entities; ordered recommendations carry a Sequence.

Example: ASP.NET Core Fundamentals is a Course; its October evening cohort is an Offering; the October 12 classroom meeting is a Session. Attendance belongs to the meeting and the learner participation in that cohort.

05 Admissions and payment ERD



Notation: PK primary key; FK foreign key; UQ unique. Relationship labels state the cardinality.

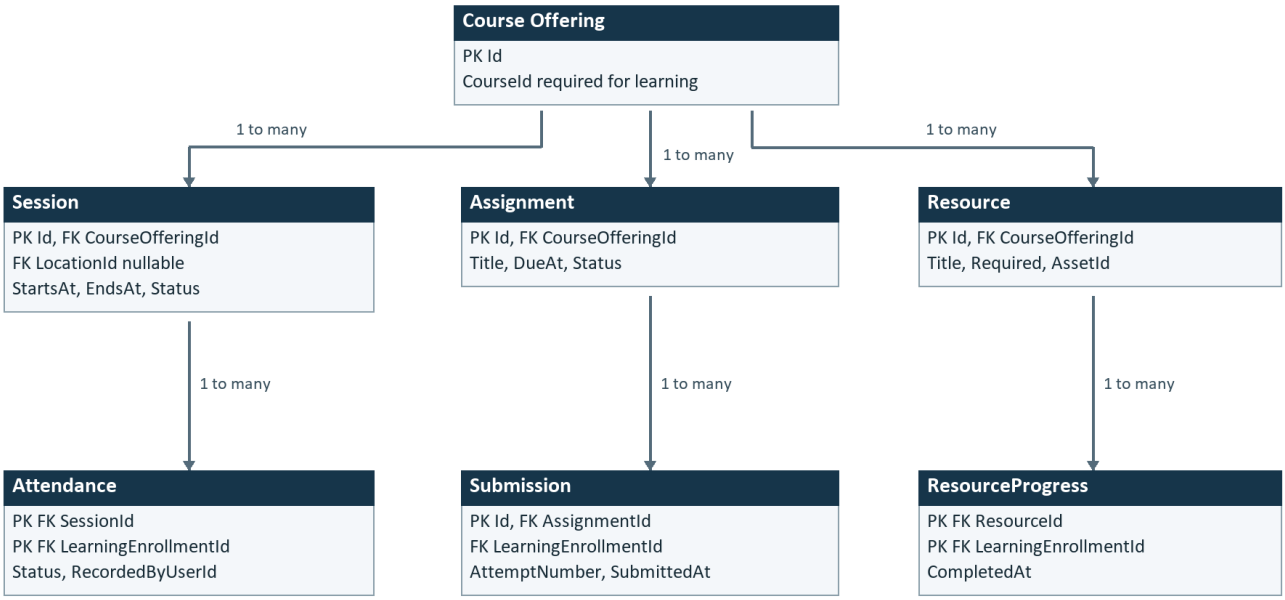
LearningEnrollment also has foreign keys to AppUser and a course Offering. These cross-links are omitted from the drawing for readability. Enrollment inherits the purchaser and purchased offering through Application; I will validate that all learning rows match that learner and the purchased offering composition.

Commercial assumptions

My Phase 1 baseline is one offering per checkout, full payment, and one order per application. Payment attempts retain retry history. A course purchase creates one learning enrollment; a program purchase creates all included course enrollments atomically. I will reject an overlapping purchase of the same active course offering until credit/transfer rules are defined.

An order stores the agreed amount and currency rather than recomputing them from the current catalog. A unique provider/payment identifier is enforced when present. Enrollment creation is idempotent through the unique ApplicationId. Financial records are retained and corrected through explicit transitions or adjustment records.

06 Learning delivery ERD



Notation: PK primary key; FK foreign key; UQ unique. Relationship labels state the cardinality.

Each bottom-row entity also belongs to one LearningEnrollment; each learning enrollment has many attendance, submission and resource-progress records. Location has many sessions. OfferingInstructor links users to assigned offerings through a composite offering/user key.

Integrity and history

- Attendance is unique per session and learning enrollment. Submission attempt numbers are unique per assignment and learning enrollment. Resource completion is unique per resource and learning enrollment.
- The session, assignment or resource must belong to the same course offering as the learning enrollment. I will use composite foreign keys with CourseOfferingId where practical, backed by application checks.
- Submission assets remain private. I will retain submission attempts and audit attendance corrections rather than overwrite operational history without attribution.
- I will model cancelled sessions explicitly. Progress calculations must define how cancelled sessions, optional resources and excused attendance affect the denominator.

07 Supporting data model

Area	Entities and key relationships
Identity	AppUser, Role, UserRole, Permission, RolePermission; InstructorProfile has one user. TermsAcceptance stores user, terms version and accepted timestamp.
Catalog discovery	Category, CourseCategory, ProgramCategory, CareerPath, CareerPathCourse, CareerPathProgram; join records carry ordering where needed.
Public content	Page and PageSection, FAQ, Partner with accreditation classification, Testimonial and MediaAsset. Page sections use defined block types and validated payloads.
Learning files	ResourceAsset and SubmissionAsset link records to object keys, original name, media type, size and scan status. Public media and private files have separate access policies.
Capacity	SeatReservation owns an application, expiry and state. ReservationItem identifies each capacity pool/offering and quantity, including all courses in a program.
Provider events	WebhookInbox stores provider/event ID, receipt time, processing status and minimal required payload. Provider/event ID is unique.
Finance	Refund or adjustment references the original payment and reason. ReconciliationCase tracks a paid-but-unfulfilled or mismatched transaction.
Compensation	InstructorAgreement belongs to an instructor and offering. EarningEntry references the agreement and snapshots rate, quantity, amount and currency. PayoutAllocation joins a payout to earning entries.
Communications	Announcement targets an offering. NotificationRecipient identifies the recipient and channel. DeliveryAttempt stores provider reference, result and retry timing.
Reliability	OutboxMessage stores durable work; JobExecution or lease fields track ownership and retry. AuditEvent records actor, action, target, timestamp and correlation ID.

Data modeling conventions

I will use explicit joins for relational business data. JSON is limited to bounded content-block payloads and integration envelopes; it will not replace foreign keys for enrollment, payments, or learning records. Supporting entities are part of the logical plan and will receive explicit mappings before their migration is merged.

The registration model includes first and last name, email, mobile, country and preferred language. Password confirmation is request validation only; password hashes and security tokens are handled by Identity. A content editor can maintain instructor biographies without receiving access to compensation records.

08 Database constraints and state models

Invariant	Planned enforcement
One target per offering	SQL check: exactly one of CourseId and ProgramId is populated.
Valid amounts and dates	Explicit decimal precision; amount and capacity bounds; start before end; valid currency and required fields.
No repeated enrollment	UQ ApplicationId on Enrollment; filtered UQ on active learner/course-offering participation. Preserve cancelled/completed history.
No duplicate processing	UQ provider/event ID; UQ idempotency key; filtered UQ provider/payment ID when non-null.
Same learner and offering	Composite alternate keys/FKs where practical; transaction-level validation of parent/child membership.
No overselling	Transactional reservation and capacity update across all required offerings; acquire rows in consistent order.
No accidental data erasure	Restrict deletes on financial, enrollment and delivery history; explicit archive/cancel states.
Concurrent edits	rowversion on mutable operational records; handle DbUpdateConcurrencyException explicitly. [5]

Separate state machines

Application: Submitted → UnderReview → Approved or Rejected; permitted withdrawal is explicit. An approved application becomes eligible for checkout. Open-admission courses may approve automatically through the same use case.

SalesOrder: PendingPayment → Paid → Fulfilled. Expired or Cancelled orders remain in history. PaymentAttempt: Initiated → Pending → Succeeded or Failed. A provider refund is recorded separately and reflected in the financial summary; a late failure event must not regress a verified success.

Enrollment and LearningEnrollment: Active → Completed or Cancelled, with suspension added only if the business requires it. Reservation: Held → Consumed, Expired or Released. Offering publication and operational availability are separate from payment and enrollment states.

Status transition contract

I will define allowed transitions in application/domain rules, validate the actor and preconditions, and write an audit event for sensitive changes. Database constraints protect structural invariants; they do not replace admission, refund or program-membership rules. Compensation approval and payment execution require independent state checks.

09 Enrollment and payment transaction design

- 1. Validate the account and offering. Persist the application and admission decision. Repeated submissions use an application-level request key or active-application rule.
- 2. Create the order with a price/currency snapshot. In a short SQL transaction reserve capacity for the offering and, for programs, every included course offering. Lock capacity rows in a stable order and persist a reservation expiry.
- 3. Commit before contacting the gateway. Create hosted checkout through an adapter using an idempotency key. Retry a timed-out provider request with the same key and reconcile its outcome. Never hold a SQL transaction open during the network call.
- 4. Verify webhook signatures against the raw payload according to the provider contract. Durably insert the unique event into WebhookInbox before acknowledging. A duplicate event receives a safe acknowledgement. [6]
- 5. Process the event idempotently. Match order, provider reference, expected amount and currency. For ambiguous or out-of-order events, query the provider or open a reconciliation case.
- 6. In one local transaction record verified payment state, consume valid capacity, create Enrollment and all LearningEnrollment rows, mark the order fulfilled, and append OutboxMessage records.
- 7. After commit, the worker delivers confirmations and enrollment notifications. The frontend polls or refreshes the backend status; its success redirect does not grant access.

Late payment and failure recovery

If a payment succeeds after reservation expiry, I will attempt to reacquire all required capacity atomically. If that fails, the payment remains recorded as successful, the order remains unfulfilled, and a reconciliation/refund case is raised. No access is granted and capacity is never silently exceeded.

If gateway checkout creation fails, release the hold when its outcome is known or let it expire. If the process crashes after database commit but before email delivery, the outbox retains the work. Delivery is at least once: handlers deduplicate effects, use bounded retries and expose exhausted attempts for operations review.

Refund boundary

The refund provider, cancellation rules and access-revocation policy must be decided before real payments launch. I will retain provider references, adjustment history and audit records. Refund success is verified separately; I will not infer it from an administrator clicking a button.

10 Security and access control

I will combine roles, named permissions, and resource-based authorization. A role establishes broad capability; the resource check verifies ownership, enrollment or instructor assignment. Every protected endpoint and file-download request performs the relevant check. [7]

Actor	Authorized scope	Key restriction
Learner	Own profile, applications, payments and active course access	Cannot select another learner ID to access records.
Instructor	Assigned course offerings, rosters, attendance and submissions	No access to unassigned cohorts or another instructor earnings.
Content editor	Published website content, catalog descriptions and biographies	No finance or unrestricted learner data access.
Manager	Operational learners, instructors, schedules and reports	Compensation requires a dedicated permission.
Administrator	Role and operational administration per permission policy	Sensitive changes remain audited; no client-side trust.

Authentication and HTTP controls

- Use Identity for password hashing, confirmation, password reset and account lockout. Use Secure, HttpOnly cookies with a deliberate SameSite policy and CSRF tokens for state-changing browser requests.
- Serve the browser and API through one origin where possible. If origins differ, allow only explicit CORS origins and review credential behavior. Share protected data-protection keys between API instances.
- Validate request DTOs and return consistent ProblemDetails responses. Distinguish validation errors, unauthenticated requests, forbidden actions, missing resources and concurrency conflicts.
- Rate-limit login, reset, application and checkout endpoints; use a shared enforcement strategy where per-instance limits would be insufficient. Store secrets outside source control.

File and content controls

Private resources and submissions use object storage with server-side authorization and short-lived download access. Uploads have size/type limits, randomized object keys, malware scanning or quarantine, and explicit ownership. Sanitize rich content and prevent unsafe HTML from entering published pages. Audit sensitive actions without logging passwords, payment secrets or unnecessary personal data.

11 Learning operations and compensation

Learning delivery

I will provide current courses and programs, applications, payment history, upcoming sessions, locations, resources, attendance and profile management. Instructors can open assigned offerings, see learner counts and permitted learner details, record attendance and review submissions. Assignment scope is submission and basic completion review; advanced grading remains deferred.

Basic progress needs a documented rule before implementation. My proposed inputs are required resource completion, attendance and assignment submission. I will version the rule and define treatment of optional items, cancelled sessions, excused absence and zero required items. I will not present submission completion as assessed mastery. Derived progress can be cached, but source records remain authoritative.

Schedules and editable content

Course offerings own the scheduled intake; sessions own meeting times and locations. Store UTC instants plus a named timezone and validate daylight-saving transitions. Session edits trigger reminders or change notices without creating duplicate jobs. Instructor and location clashes require validation according to agreed scheduling rules.

Editors can maintain homepage sections, About Us, course/program/career-path details, instructor biographies, partners, FAQs, contact/support information, images and publication status. Schedule and location edits require operational permission and audit. Draft content must not leak through public search or cached pages.

Compensation and reporting

I will start with an agreed fixed-fee or hourly instructor agreement. Earning entries snapshot the applicable rate, quantity, currency and source event. Later agreement changes do not recalculate history. Payout allocations connect approved earnings to recorded payouts and must prevent over-allocation or cross-currency mixing.

The dashboard will distinguish earned, approved and paid totals. Revenue will have a defined basis, such as verified collected payments less verified refunds for a period, and must not be confused with recognized accounting revenue. Active students, active instructors and running courses also need explicit definitions and an as-of timestamp.

Notification coverage

I will cover account confirmation, application submission/status, payment and enrollment confirmation, session reminders, announcements, instructor assignment, new enrollment, assignment submission and operational notices. Email is the initial channel assumption. Each delivery records recipient, event, template, status and retry history. Notifications are generated after durable business changes and deduplicated by recipient/event/channel.

12 EF Core and migration strategy

I will use code-first entities with explicit Fluent API configurations. Migrations and model snapshots are version-controlled. The database model is reviewed alongside the domain model; generated SQL is an engineering deliverable, not an unchecked side effect.

Concern	Implementation convention
Identifiers	bigint business keys; compatible Identity key strategy selected at foundation. Opaque public references may be used but never replace authorization.
Columns	Explicit required/nullability, string lengths, decimal precision and datetimeoffset mappings. Store currency with every monetary amount.
Concurrency	SQL rowversion on mutable operational aggregates; refresh/retry only for safe commands, otherwise return a conflict. [5]
Queries	DTO projections, no tracking for reads, cancellation tokens, bounded results and explicit loading. DbContext is scoped and not shared concurrently.
Transactions	Short SQL transactions around invariants. Coordinate execution-strategy retries with idempotent workflows. Keep external calls outside the transaction.
History	Restrict cascading deletes for financial/enrollment records; archive catalog entries and audit sensitive edits.

Migration delivery workflow

- Change the model and mappings, generate a named migration, and review SQL for destructive operations, defaults, index cost, lock duration and unintended drop/recreate behavior.
- Test creation from an empty SQL Server database and upgrade from the previous release with representative data. Validate constraints and application compatibility after the upgrade.
- Produce a reviewed SQL script or migration bundle. Apply once in the deployment pipeline with a dedicated migration identity; the normal API identity does not require schema-change privileges.
- Use expand → backfill → switch → remove for breaking changes. Keep old and new application versions compatible during rollout. Backfill large datasets in bounded batches.
- Do not edit a migration already applied to a shared environment. Resolve migration-stream conflicts before merge. Use a forward fix or tested recovery plan rather than assuming a destructive down migration is safe.

Production migrations will not run as routine API startup work. The deployment process validates the target migration state and backup/recovery readiness. Microsoft recommends inspecting and testing migrations before production application. [8]

13 Query performance and scale strategy

I will measure SQL and API behavior with realistic data before adding caching or infrastructure. The baseline avoids lazy loading, projects only required DTO fields, paginates growing collections and inspects query plans for slow paths. Keyset pagination suits long sequential lists; offset pagination remains acceptable for bounded page-number navigation. [9]

Query or invariant	Initial index candidate
Learner applications	Application (LearnerId, SubmittedAt, Id)
Offering roster	LearningEnrollment (CourseOfferingId, Status, LearnerId)
Upcoming sessions	Session (CourseOfferingId, StartsAt)
Payment reconciliation	PaymentAttempt (Status, CreatedAt)
Pending work	Filtered index on pending outbox/inbox work with NextAttemptAt
Duplicate access	Filtered unique learner/course-offering index for active participation
Attendance and submissions	Composite uniqueness matching the business keys in the LMS ERD

Index definitions are candidates until validated with execution plans and production-shaped data. I will account for write overhead, selective filters, stable sort tie-breakers and included columns. Basic search will have a bounded query strategy; I will not assume a leading-wildcard search benefits from a normal B-tree index.

Caching and background work

Cache published catalog content with publish-time invalidation and a fallback TTL. Keep authorization, payment decisions and capacity authoritative in SQL Server. Add a shared cache only when the measured workload warrants it. Large exports, reminders, email and reconciliation run asynchronously. Dashboard summaries may be precomputed with a visible freshness timestamp.

Benchmark targets

Measure	Initial validation target
Ordinary API reads	p95 below 300 ms
Internal writes	p95 below 500 ms, excluding gateway latency
Concurrency correctness	Zero duplicate enrollment and zero overselling
Benchmark disclosure	Hardware, SQL tier, dataset, request mix, concurrency, duration, throughput, error rate and cache state recorded

These are proposed targets. I will establish the expected user count and peak workload before treating them as release gates. I will publish measured results and bottlenecks rather than claiming untested scalability.

14 Deployment and operational readiness

Deployment topology

The initial topology is React hosting, API, worker, SQL Server and object storage. Multiple API instances can sit behind a load balancer. API processes remain stateless with shared authentication keys; uploads, queue state and work leases stay off local disk. Worker instances claim durable jobs with leases and recover expired claims after a crash.

I will keep development, staging and production configuration separate. CI builds the frontend and backend, checks types and formatting, runs relevant tests, scans dependencies, and packages versioned deployable artifacts. CD applies reviewed migrations, deploys a compatible version and runs smoke checks. Deployment rollback must account for database compatibility.

Observability

- Structured logs with correlation IDs across API requests, payment events, database workflow and notification jobs. Redact sensitive values.
- Metrics for latency, request errors, SQL duration, connection pressure, checkout failures, capacity conflicts, inbox/outbox age and exhausted retries.
- Traces around important use cases and provider calls. Dashboard/report failures must be distinguishable from enrollment failures.
- Readiness and liveness endpoints plus alerts for payment/enrollment mismatch, growing job backlog and failed backups.

Recovery and operational procedures

I will document how to replay failed inbox/outbox items safely, reconcile provider transactions, resolve paid-but-unfulfilled orders, recover expired worker leases and correct attendance or payouts with an audit trail. Scheduled jobs must handle rescheduling and cancellation without duplicate reminder delivery.

Database backups and file retention need a documented recovery point objective and recovery time objective. These values remain to be agreed. Before launch I will perform a restore exercise and verify that enrollment records, private file references and financial history remain consistent. Access to production secrets and operational tools is restricted by role.

Scaling sequence

My order of optimization is query shape and indexes, then API/worker instance counts, then targeted distributed caching or report precomputation. I will consider read replicas or extracting a module only when a measured workload or team boundary justifies the added consistency and operational complexity. Multi-tenancy is not assumed for Phase 1.

15 Implementation milestones

Milestone	Implementation output	Acceptance evidence
1 Business rules	State diagrams, permissions matrix, logical/physical schema decisions and API contracts	Course/program purchase, failure handling, progress and compensation rules reviewed.
2 Foundation	Identity, authorization policies, DbContext, migrations, CI, logging, storage and worker skeleton	Account flow works; unauthorized access fails; clean database creation succeeds.
3 Catalog and CMS	Public pages, courses, programs, career paths, categories, offerings and publication workflow	Editors publish; visitors discover and compare actual intakes; drafts stay private.
4 Enrollment slice	Applications, reservations, orders, checkout, webhook inbox, enrollment and outbox	Visitor-to-resource journey survives retries and concurrent final-seat checkout.
5 Learning delivery	Sessions, rosters, resources, attendance, assignments and progress	Learner and assigned instructor complete end-to-end learning workflows.
6 Operations	KPIs, reports, compensation, payouts, content completion and notifications	Financial totals reconcile; role boundaries and required notices are verified.
7 Hardening	Load testing, accessibility, recovery, migration upgrade and deployment checks	Reproducible benchmark, restore evidence and production readiness review.

Dependency and estimation approach

Milestones are ordered by dependency rather than calendar promises. Identity and offering definitions precede the enrollment slice; stable enrollment precedes LMS permissions and roster operations. Supporting CMS work can be developed around agreed catalog contracts. I will estimate dates after confirming the deadline, team capacity, supplied design assets, gateway and language requirements.

Definition of done

A feature is done when its business rules and API contract are implemented, authorization is enforced, relevant SQL constraints and migrations are reviewed, failure/empty/loading states are usable, appropriate automated checks pass, and operational evidence is available. Each milestone ends with a demonstrable user journey and an updated decision log.

16 Verification and competition evidence

I will use focused unit tests for domain transitions and progress rules, SQL Server integration tests for relational behavior, API tests for permissions and contracts, and browser tests for critical journeys. EF InMemory is not sufficient evidence for SQL Server constraints, transactions, collation, filtered indexes or concurrency.

Scenario	Expected evidence
Two users buy the final seat	Exactly one capacity allocation succeeds; the other receives a defined outcome without overselling.
Program purchase with one full course	All required seats are reserved or none are; no partial program access is created.
Repeated checkout request	Provider idempotency and application constraints prevent accidental duplicate processing.
Duplicate or out-of-order webhook	One enrollment effect; verified success does not regress due to an older event.
Crash after enrollment commit	Outbox survives; confirmation resumes without duplicating business records.
Payment after reservation expiry	Reacquire capacity or create a paid-but-unfulfilled case; no silent over-enrollment.
Unauthorized identifier substitution	Learner/instructor cannot access another user or unassigned offering by changing IDs.
Mismatched LMS relationship	Attendance/submission against a different course offering is rejected.
Concurrent schedule or attendance edit	rowversion conflict is surfaced or safely resolved without lost updates.
Compensation rate change	Existing earning amounts remain unchanged; payouts cannot over-allocate earnings.
Database upgrade and restore	Previous-release data upgrades successfully; restored data and file links remain usable.
Load and frontend accessibility	Latency/error evidence under declared load; keyboard navigation, labels and form errors verified.

Review package

I will present the architecture and ERDs, a short decision log explaining tradeoffs, an automated test summary, migration/restore evidence, and a reproducible performance report. The demonstration will include failure recovery and authorization checks as well as the successful enrollment journey. This provides evidence for correctness, maintainability and scalability.

17 Decisions to confirm

The following assumptions define my starting implementation baseline. I will resolve them before finalizing the affected schema, acceptance criteria or provider integration. They are proposed rules, not approved stakeholder decisions.

Decision	Working baseline	Impact if changed
Admission timing	Approval before payment; automatic approval for open courses	Changes checkout eligibility and status transitions.
Payment model	One offering per order and full payment	Cart, installments or partial settlement require a broader order/allocation model.
Gateway and refund policy	Provider not selected; verified events required	Determines event contract, reconciliation and cancellation behavior.
Program composition	Fixed included course offerings per intake	Electives need selection and reservation rules.
Overlapping purchases	Block duplicate active course-offering participation	Credits or shared entitlements need a revised access-source model.
Reservation duration	Time-limited hold; duration to be agreed	Affects checkout expiry and late-payment recovery.
Assignments and progress	Submission/basic review; versioned completion rule	Changes progress thresholds and required LMS fields.
Notifications	Email first	SMS, push or in-app channels add adapters and delivery requirements.
Language and RTL	Store preferred language; launch locales undecided	Affects content translations, routing and UI verification.
Compensation	Fixed fee or hourly agreement	Revenue shares need a defined allocation and adjustment basis.
Academy structure	Single academy	Corporate accounts or tenancy require access-boundary redesign.
Capacity and schedule policy	Program and constituent course capacity enforced	Clarify shared cohorts, transfers and instructor/location conflicts.
Operational definitions	KPI basis, retention, peak load and recovery objectives to agree	Sets reports, data lifecycle, test load and release targets.

The source scope also calls for stakeholder confirmation of Phase 1 boundaries, offline-course launch focus, permissions and editable content. I will resolve those through normal scope review; they do not imply that this implementation plan has already been signed off. [1]

18 Technical references and decision record

These references support the technical choices in this plan. Provider documentation will be checked again when the integration is selected, and runtime patch versions will be verified when implementation begins.

1 Andalusia Academy Phase 1 MVB Website and LMS Feature Scope

Review Draft version 1.0, 19 August 2026. Pages 4–8 define launch features; page 9 lists deferred scope; page 12 lists stakeholder decisions. Source file: Andalusia_Academy_Phase_1_MVB_Scope.pdf.

2 Microsoft common web application architectures

https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures

3 Official NET support policy

https://dotnet.microsoft.com/en-us/platform/support/policy

4 React application creation guidance

https://react.dev/learn/creating-a-react-app

5 EF Core concurrency conflicts

https://learn.microsoft.com/en-us/ef/core/saving/concurrency

6 Stripe webhook integration guidance

https://docs.stripe.com/webhooks — reference example for signatures, duplicate events and delivery behavior; Stripe is not a selected provider.

7 ASP NET Core resource based authorization

https://learn.microsoft.com/en-us/aspnet/core/security/authorization/resource-based?view=aspnetcore-10.0

8 EF Core applying migrations

https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/applying

9 EF Core efficient querying

https://learn.microsoft.com/en-us/ef/core/performance/efficient-querying

Initial architecture decisions

ADR 001: modular monolith with explicit module ownership. ADR 002: course definitions separated from scheduled offerings. ADR 003: independent application, payment and enrollment lifecycles. ADR 004: transactional capacity and program access. ADR 005: durable inbox/outbox with idempotent handlers. ADR 006: reviewed code-first migrations through deployment. ADR 007: resource authorization and private object storage. ADR 008: performance claims supported by reproducible measurements.